



Committed to Excellence

DEPARTMENT OF SOFTWARE TECHNOLOGY

Software Engineering Project

SecurePay

PROJECT MANUAL

A secure digital payment and transaction management system

Team ID: SET-3A-17

TEAM MEMBERS

Farhan Shafi

GR: 2024-DSET/M-004

Sarim Mahmood

GR: 2024-DSET/M-002

GUIDED BY

Sir Nehal Naveed

Contents

1. Introduction
2. Objectives
3. Technology Stack
4. System Architecture
5. Database Design
6. Key Features
7. How It Works
8. Fraud Detection
9. Security & Reliability
10. Installation & Running
11. Conclusion & Future Scope

1. Introduction

SecurePay is a secure, microservices-based digital wallet. Users register and verify their email, then open one or more wallets — one per currency — fund them, save beneficiaries, and send money to anyone, including across currencies at the live exchange rate. They can also receive money by sharing a QR code, pay bills, and review a full statement they can export as a PDF.

Safety is built into every step: transfers are debited and credited inside a single locked database transaction, screened for fraud in real time before any money moves, and confirmed with the phone's fingerprint or face. The backend is a set of Python (FastAPI) microservices running in Docker, backed by PostgreSQL, Redis and RabbitMQ; the client is a React Native (Expo) app that runs on both Android and iOS.

2. Objectives

- To automate digital payments with a secure, easy-to-use multi-currency wallet.
- To guarantee correct money handling using ACID transactions and row locking, so a balance can never be corrupted or a payment duplicated.
- To detect and block fraudulent transfers in real time, before any money moves.
- To support multiple currencies with automatic conversion at live exchange rates.
- To secure accounts and every payment with email verification and device biometrics.
- To reliably notify users of activity, and to demonstrate a scalable microservices design.

3. Technology Stack

The project uses a modern, industry-standard stack. Each tool was chosen for a specific reason:

Layer	Technology	Why it was used
Mobile app	React Native, Expo, TypeScript	One codebase for Android + iOS; camera (QR) and biometrics built in
Backend	Python, FastAPI	Fast to build, auto-generated API docs, high performance
Database	PostgreSQL	ACID transactions + row locking keep money correct
Cache / OTP	Redis	Fast in-memory store for rate limits and expiring codes
Message broker	RabbitMQ	Async events so notifications never slow down a payment
Security	JWT (access + refresh), bcrypt	Stateless auth with silent refresh; safely hashed passwords
Email	Resend	Delivers verification codes and transaction alerts
DevOps	Docker, Compose, GitHub Actions	One-command run; CI checks every push
Tools	VS Code, Git & GitHub	Development environment and version control

4. System Architecture

SecurePay follows a microservices architecture: instead of one large program, the system is split into small services that each own one job. All requests enter through a single API Gateway, which rate-limits traffic and forwards each request to the correct service. The services share one PostgreSQL database, use Redis for temporary data, and communicate asynchronously through RabbitMQ.

Request path:

Mobile App → API Gateway → Microservices → PostgreSQL

Component	Responsibility
API Gateway	Single public entry point; per-IP rate limiting (Redis); routing
User Service	Register, login (JWT + refresh), verify email, forgot password, notifications
Wallet Service	Create wallets (one per currency), balance, deposit, statement, beneficiaries
Transaction Service	P2P transfers and bill payments, live FX, publishes events via an outbox
Fraud Service	Real-time rule-based screening of every transfer (internal)
Notification Service	Background worker; consumes events and sends email alerts

5. Database Design

The schema is versioned with Alembic migrations. The main entities and their key fields are:

Entity	Key fields	Purpose
users	user_id (PK), email, phone, password_hash, kyc_verified	Account details
wallets	wallet_id (PK), user_id (FK), currency, balance	A user's money (one row per currency)
transactions	transaction_id (PK), sender/recipient (FK), amount, fx_rate, idempotency_key	Every transfer / bill
beneficiaries	id (PK), owner_user_id (FK), wallet_id (FK), nickname	Saved payees
fraud_logs	id (PK), wallet_id (FK), fraud_score, risk_level, action_taken	Flagged / blocked transfers
notifications	id (PK), user_id (FK), channel, message, status	Sent alerts (audit trail)
billers	id (PK), name, category, wallet_id (FK)	Bill payees (system wallets)
outbox_events	id (PK), routing_key, payload, status	Reliable event delivery (outbox)

Relationships: a user has many wallets (one per currency); a wallet has many transactions; a user has many beneficiaries, fraud logs and notifications; each biller is backed by a system wallet.

6. Key Features

- **Multi-currency wallets** — hold USD, EUR, GBP and PKR at once — one wallet per currency, switched from the home screen.
- **Email verification (OTP)** — a 6-digit code is emailed at sign-up; a wallet can only be opened after the email is verified.
- **Send & receive by QR** — share your wallet as a QR code, or scan someone else's to pay them without typing an id.
- **Live currency conversion** — cross-currency transfers are converted at the live rate, previewed before you send.
- **Real-time fraud detection** — a rule-based engine scores every transfer and blocks suspicious ones.
- **Bill payments** — pay electricity, internet, gas and mobile billers by reference number.
- **Biometric confirmation** — every payment is confirmed with the device fingerprint or face.
- **Beneficiaries** — save payees once and send to them by name.
- **Statements, PDF & insights** — filter transactions by date, export a PDF, and view a monthly money-in/out chart.
- **Notifications & security center** — an in-app feed of alerts, plus a screen showing your own fraud-screening events.
- **Forgot password** — reset a lost password with a code emailed to the account address.

7. How It Works

A typical user journey flows through the system as follows:

- **Register & verify:** the user signs up; the user-service emails a 6-digit code (stored in Redis for 5 minutes) which the user enters to verify their email.
- **Create wallet & deposit:** once verified, the user opens a wallet in a chosen currency (and more later, one per currency) and adds money.
- **Pay:** the user sends money to a payee (typed, saved, or QR-scanned) or pays a bill. After a biometric check, the transaction-service screens it for fraud, converts the amount if needed, then debits the sender and credits the recipient in one atomic, locked commit.
- **Notify & record:** the transfer writes an event to an outbox in the same commit; a drainer publishes it to RabbitMQ, the notification-service emails both parties, and the statement records it.

8. Fraud Detection

Every transfer is screened before any money moves. The fraud-service is rule-based: each rule that fires adds points to a risk score.

Rule	Fires when	Points
High velocity	More than 5 transfers from a wallet in the last 5 minutes	+30
Unusual amount	The transfer is more than 2x the wallet's average transfer	+25
New large recipient	First-ever payment to this recipient and it is $\geq$ \$500	+20
Statistical anomaly	The amount is 3+ standard deviations above the wallet's own history	+15

The total score maps to a risk level and an action:

Score	Risk level	Action
0 - 29	Low	Approve
30 - 59	Medium	Review (allowed, but logged)
60 - 84	High	Block (HTTP 403)
85 - 100	Critical	Block (HTTP 403)

Example: a user makes 6 rapid transfers, then sends \$600 to a brand-new recipient. Three rules fire: $30 + 25 + 20 = 75$ (High) — the transfer is blocked. The statistical-anomaly rule is an ML-lite stepping stone: it adapts to each user by scoring against their own spending pattern (a z-score), the first feature a real machine-learning fraud model would use.

9. Security & Reliability

- **Password hashing (bcrypt):** passwords are never stored in plain text — only a slow, salted hash.
- **Stateless auth (JWT + refresh):** a short-lived access token proves identity; the app silently exchanges a refresh token for a new one, so sessions last without long-lived tokens.
- **Email verification & biometrics:** the user proves they own their email, and confirms every payment with the device fingerprint or face.
- **Row locking & ACID:** transfers lock both wallets (in a fixed order) so concurrent transfers can't corrupt a balance.
- **Idempotency keys:** the same transfer sent twice is executed only once.
- **Rate limiting:** the gateway caps requests per IP to resist abuse.
- **Transactional outbox:** each event is written in the same database transaction as the transfer, then published to RabbitMQ by a drainer — so a completed transfer can never silently lose its notification (at-least-once delivery).

10. Installation & Running

Prerequisites: Docker Desktop (includes Docker & Docker Compose), and Node.js with the Expo Go app on a phone for the mobile client.

Run the backend

1. `cd securepay`
2. `cp .env.example .env`
3. `docker compose up --build`
4. `python3 scripts/smoke_test.py` (end-to-end check)

Run the mobile app

1. `cd securepay/mobile`
2. `npm install`
3. `npx expo start` (scan the QR code with Expo Go on the same Wi-Fi)

To stop: `docker compose stop` (data is preserved). To start again: `docker compose start`. Never use `docker compose down -v` — it deletes all data.

11. Conclusion & Future Scope

SecurePay delivers a secure, scalable and user-friendly digital wallet. Its microservices design, real-time fraud screening, multi-currency support, bill payments and biometric-protected transfers bring a production-grade fintech experience to a single platform. The project demonstrates practical use of Docker, PostgreSQL, Redis, RabbitMQ, JWT security, a transactional outbox, and cross-platform mobile development.

Future scope: a machine-learning fraud model on top of the current signals, push notifications, merchant / shop QR payments, a web version, and debit-card integration.